

```
[Column("3")]
public float PetalWidth;

[Column("4")]
[ColumnName("Label")]
public string Label;
}
```

For the prediction results, use the following class:

```
public class IrisPrediction
{
    [ColumnName("PredictedLabel")]
    public string PredictedLabels;
}
```

The core functionality is coded in the following function:

```
static void Main(string[] args)
{
    // STEP 2: Create a pipeline and load your data

    var pipeline = new LearningPipeline();

    // If working in Visual Studio, make sure the 'Copy to Output
    // Directory' // property of iris-data.txt is set to 'Copy always'

    string dataPath = "iris-data.txt";
    pipeline.Add(new TextLoader(dataPath).
    CreateFrom<IrisData>(separator: ','));

    // STEP 3: Transform your data // Assign numeric values to
    // text in the "Label" column, because only // numbers can be
    // processed during model training

    pipeline.Add(new Dictionary("Label"));

    // Puts all features into a vector

    pipeline.Add(new ColumnConcatenator("Features",
    "SepalLength", "SepalWidth", "PetalLength", "PetalWidth"));

    // STEP 4: Add learner
    // Add a learning algorithm to the pipeline.
    // This is a classification scenario (What type of iris is
    // this?)

    pipeline.Add (new StochasticDualCoordinateAscentClassifier());

    // Convert the Label back into original text (after converting
    // to number in step 3)

    pipeline.Add(new PredictedLabelColumnOriginalValueConverter())
```

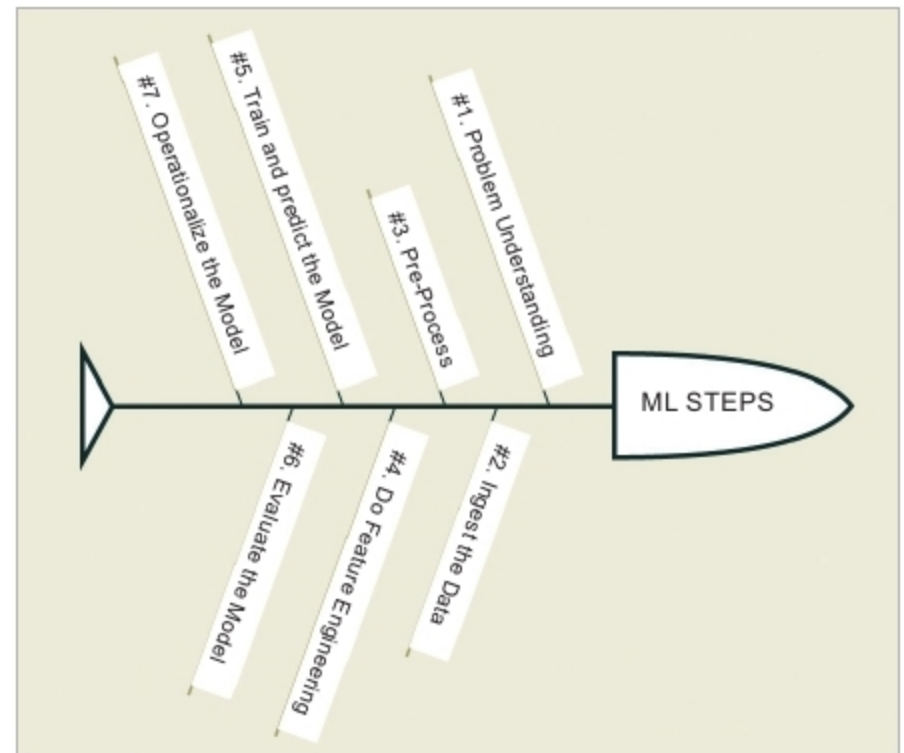


Figure 3: Using ML.NET to build a ML solution

```
{ PredictedLabelColumn = "PredictedLabel" });

// STEP 5: Train your model based on the data set

var model = pipeline.Train<IrisData, IrisPrediction>();

// STEP 6: Use your model to make a prediction // You can
// change these numbers to test different predictions

var prediction = model.Predict(new IrisData() { SepalLength
= 3.3f, SepalWidth = 1.6f, PetalLength = 0.2f, PetalWidth =
5.1f, });

Console.WriteLine($"Predicted flower type is: {prediction.
PredictedLabels}");
}
```

- Step 6: The app can be run with the following code:

```
>dotnet run
```

There are many ML.NET tutorials available on the official website (<https://docs.microsoft.com/en-gb/dotnet/machine-learning/tutorials/>), including:

- Sentiment analysis
- Iris clustering
- Taxi fare predictor

The major steps that need to be taken to solve most of the problems with ML.NET are:

- Understand the problems
- Ingest the data
- Pre-process the data and do feature engineering
- Train the model to make predictions
- Evaluate the model
- Operationalise the model